

PATRÓN ITERADOR

Capítulo 16 — Patrones de Diseño en Java

Traducción técnica al español

1. Descripción

El patrón Iterador permite que un objeto cliente acceda secuencialmente al contenido de un contenedor, sin necesidad de conocer los detalles internos de su representación. El término «contenedor» puede definirse simplemente como una colección de objetos, los cuales pueden ser a su vez colecciones anidadas. El patrón Iterador habilita a un objeto cliente para recorrer dicha colección sin exponer cómo los datos son almacenados internamente.

Para lograr esto, el patrón sugiere que el Contenedor debe diseñarse para proveer una interfaz pública en forma de Iterador, permitiendo que distintos clientes accedan a su contenido. Un objeto Iterador expone métodos públicos que permiten al cliente navegar a través de la lista de objetos dentro del contenedor.

2. Iteradores en Java

Uno de los iteradores más simples disponibles en Java es la clase `java.sql.ResultSet`, que se utiliza para almacenar registros de bases de datos. Esta clase ofrece un método para navegar entre filas y un conjunto de métodos getter para el acceso a columnas.

Java también provee la interfaz `Enumeration` como parte del paquete `java.util`, que declara los siguientes métodos:

Método	Retorno	Descripción
<code>hasMoreElements()</code>	<code>boolean</code>	Verifica si existen más elementos en la colección.
<code>nextElement()</code>	<code>Object</code>	Retorna el siguiente elemento de la colección.

Tabla 16.1 — Métodos de la interfaz `Enumeration`

Adicionalmente, la interfaz `java.util.Iterator` declara tres métodos:

Método	Retorno	Descripción
<code>hasNext()</code>	<code>boolean</code>	Verifica si existen más elementos en la colección.
<code>next()</code>	<code>Object</code>	Retorna el siguiente elemento de la colección.

Método	Retorno	Descripción
remove()	void	Elimina de la colección el último elemento retornado por el iterador.

Tabla 16.2 — Métodos de la interfaz Iterator

Aunque es recomendable utilizar las interfaces de iteración existentes en Java (Iterator o Enumeration), no es obligatorio hacerlo. Es posible diseñar una interfaz personalizada más adecuada a las necesidades específicas de una aplicación.

3. Iteradores Filtrados

En el caso de la clase `java.util.Vector`, su iterador simplemente retorna todos los elementos de la colección en orden secuencial. Sin embargo, un iterador puede implementarse para retornar únicamente un subconjunto de objetos seleccionados, en lugar de la totalidad de elementos. Este filtrado puede basarse en parámetros proporcionados por el cliente. Este tipo de iteradores se denominan iteradores filtrados.

4. Iteradores Internos vs. Externos

Un iterador puede diseñarse como interno o como externo, con las siguientes características diferenciadas:

4.1 Iteradores Internos

- El propio contenedor ofrece los métodos que permiten al cliente recorrer los objetos que contiene. Por ejemplo, la clase `java.util.TreeSet` contiene los datos y también expone métodos para navegar por la lista.
- Solo puede existir un iterador sobre la colección en un momento dado.
- El contenedor debe mantener y preservar el estado de la iteración.

4.2 Iteradores Externos

- La funcionalidad de iteración se desacopla del contenedor y reside en un objeto separado denominado iterador. Generalmente, el contenedor retorna un objeto iterador apropiado al cliente según los parámetros recibidos. Por ejemplo, la clase `java.util.Vector` delega su iteración en un objeto separado de tipo `Enumeration`, retornado mediante el método `elements()`.
- Pueden existir múltiples iteradores sobre una misma colección de manera simultánea.
- La sobrecarga de mantener el estado de la iteración no recae sobre el contenedor, sino sobre el objeto iterador exclusivo.

5. Ejemplo: Iterador Interno

Se construye una aplicación para mostrar datos de un archivo `Candidate.txt`, que contiene información de profesionales de TI que han enviado su perfil para una vacante. Para simplificar, se consideran tres atributos: nombre, ubicación actual y tipo de certificación.

Se define una clase contenedora `AllCandidates` (Listado 16.1) que:

- Lee los datos del archivo como parte de su constructor y los almacena como objetos `Candidate` dentro de una instancia de tipo `Vector`.
- Implementa la interfaz built-in `java.util.Iterator` y provee implementación para sus métodos: `hasNext()`, `next()` y `remove()`.

5.1 Listado 16.1 — Clase `AllCandidates`

```
public class AllCandidates implements Iterator {
    private Vector data;
    Enumeration ec;
    Candidate nextCandidate;

    public AllCandidates() {
        initialize();
        ec = data.elements();
    }

    public boolean hasNext() {
        nextCandidate = null;
        while (ec.hasMoreElements()) {
            Candidate tempObj = (Candidate) ec.nextElement();
            nextCandidate = tempObj;
            break;
        }
        return (nextCandidate != null);
    }

    public Object next() {
        if (nextCandidate == null) throw new NoSuchElementException();
        return nextCandidate;
    }

    public void remove() {}
}
```

5.2 Interacción Cliente / Contenedor

El cliente SearchManager crea la interfaz de usuario necesaria para mostrar los datos. Al hacer clic en el botón «Mostrar Todos», se instancia el contenedor AllCandidates. Como parte de su constructor, el objeto AllCandidates lee el archivo de datos y almacena la información internamente como objetos Candidate dentro de un Vector. El cliente no necesita conocer de qué forma o en qué estructura se almacenan los datos.

Para el cliente, el objeto AllCandidates actúa simultáneamente como contenedor e iterador. Utiliza los métodos hasNext() y next() para acceder a los distintos objetos Candidate y presentarlos en la interfaz de usuario.

6. Ejemplo: Iterador Externo Filtrado

Se mejora la aplicación para permitir que el usuario filtre candidatos según el tipo de certificación. Esta mejora se diseña mediante un iterador externo filtrado. En el caso del iterador externo, la implementación está desacoplada del contenedor y reside en una clase iteradora separada.

Se diseña la clase iteradora externa CertifiedCandidates como implementadora de java.util.Iterator. En su constructor, acepta un tipo de certificación y una instancia de AllCandidates. Implementa los métodos del iterador de la siguiente manera:

- hasNext(): Verifica si existen más candidatos con el tipo de certificación especificado.
- next(): Retorna el siguiente candidato con la certificación especificada. Si no existe ninguno, lanza una excepción NoSuchElementException. Idealmente, el cliente debe invocar next() únicamente si una llamada previa a hasNext() retornó true.
- remove(): Dado que la aplicación no contempla la eliminación de perfiles de candidatos, este método no realiza ninguna acción.

6.1 Listado 16.2 — Clase CertifiedCandidates

```
public class CertifiedCandidates implements Iterator {
    AllCandidates ac;
    String certificationType;
    Candidate nextCandidate;
    Enumeration ec;

    public CertifiedCandidates(AllCandidates inp_ac, String certType) {
        ac = inp_ac;
        certificationType = certType;
        ec = inp_ac.getAllCandidates();
    }

    public boolean hasNext() {
        boolean matchFound = false;
        while (ec.hasMoreElements()) {
            Candidate tempObj = (Candidate) ec.nextElement();
```

```

        if
        (tempObj.getCertificationType().equals(certificationType)) {
            matchFound = true;
            nextCandidate = tempObj;
            break;
        }
    }
    if (!matchFound) nextCandidate = null;
    return matchFound;
}

public Object next() {
    if (nextCandidate == null) throw new NoSuchElementException();
    return nextCandidate;
}

public void remove() {}
}

```

6.2 Listado 16.3 — Clase AllCandidates Modificada

La clase AllCandidates se modifica para exponer los métodos necesarios que permiten al iterador externo acceder a los datos:

```

public class AllCandidates {
    private Vector data;

    public Enumeration getAllCandidates() {
        return data.elements();
    }

    public Iterator getCertifiedCandidates(String type) {
        return new CertifiedCandidates(this, type);
    }
}

```

7. Beneficio Clave del Diseño

En ambos casos (iterador interno y externo), el cliente SearchManager no contiene ninguna implementación vinculada a la forma en que los datos se almacenan dentro del contenedor. Toda esa lógica se traslada al contenedor (iterador interno) o al objeto iterador (iterador externo). Este diseño protege al cliente SearchManager de cualquier cambio en la representación interna de los datos. Por ejemplo, si los datos dejan de almacenarse en un Vector y pasan a usarse un array u otra estructura, no se requieren modificaciones en el cliente.

8. Preguntas de Práctica

1. Diseñar e implementar un nuevo iterador externo filtrado que filtre candidatos por ubicación e integrarlo en la aplicación de ejemplo.
2. Considerar el siguiente archivo de datos XML de autores y diseñar una aplicación que recorra la lista utilizando los componentes descritos en la Tabla 16.3:

Nombre	Rol	Responsabilidad
AuthorCollection	Contenedor	Lee el archivo XML físico y almacena los datos. Ofrece métodos para crear dos iteradores externos: AuthorIterator y BookIterator.
AuthorIterator	Iterador	Iterador externo que retorna autores uno a uno en respuesta a llamadas al método next().
BookIterator	Iterador filtrado	Iterador externo filtrado que retorna los libros escritos por un autor especificado, uno a uno, en respuesta a llamadas al método next().

Tabla 16.3 — Componentes de la aplicación de práctica